

Spring AI 및 Langchain4j를 활용한 생성형 AI 및 RAG 지원



Contents

1. 생성형 AI 개발,
Spring AI로 시작하기
2. RAG 적용 방안 소개
3. Langchain4j 와
Spring AI 비교
4. Spring AI 기반 RAG
Chatbot 구현 사례



1. 생성형 AI 개발, Spring AI로 시작하기

1. 기본 개념
2. Spring AI란
3. 주요 특징 및 기능
4. 표준프레임워크의 Spring AI 공식 지원

□ LLM (Large Language Model)

– LLM 이란

- 대규모 텍스트 데이터로 학습된 인공지능 모델로, 자연어를 이해하고 생성할 수 있다.
- 번역, 요약, 질의응답, 코드 생성 등 다양한 작업에 응용 가능하다.
- 상용 외에도 오픈 소스로도 배포되고 있어 로컬 환경에서 사용도 가능하다.

– 대표적인 LLM 비교

모델 / 계열	특징	강점
GPT 계열 (OpenAI)	범용 성능 우수	추론
Claude 계열 (Anthropic)	장문 처리 강점	요약, 보고서 작성
Gemini 계열 (Google)	멀티모달 지원	이미지, 텍스트 통합
Llama 계열 (Meta)	대표 오픈 모델	로컬 구축, 확장성
Mistral 계열	경량, 고속	효율적 배포

❑ Ollama

– Ollama

- 로컬 환경에서 오픈 소스 LLM을 쉽게 다운로드, 실행, 관리할 수 있게 해 주는 도구이다.
- vLLM, llama.cpp 계열의 다른 도구들과 비교하여 설치 및 사용이 간단하므로 로컬 PC에서 오픈소스 LLM을 빠르게 실행하고 테스트하는 용도에서 이점을 가진다.

– 기본 사용법

```
# 모델 다운로드
ollama pull <model-name>

# Hugging Face에서 모델 다운로드
ollama pull hf.co/<username>/<model-repository>

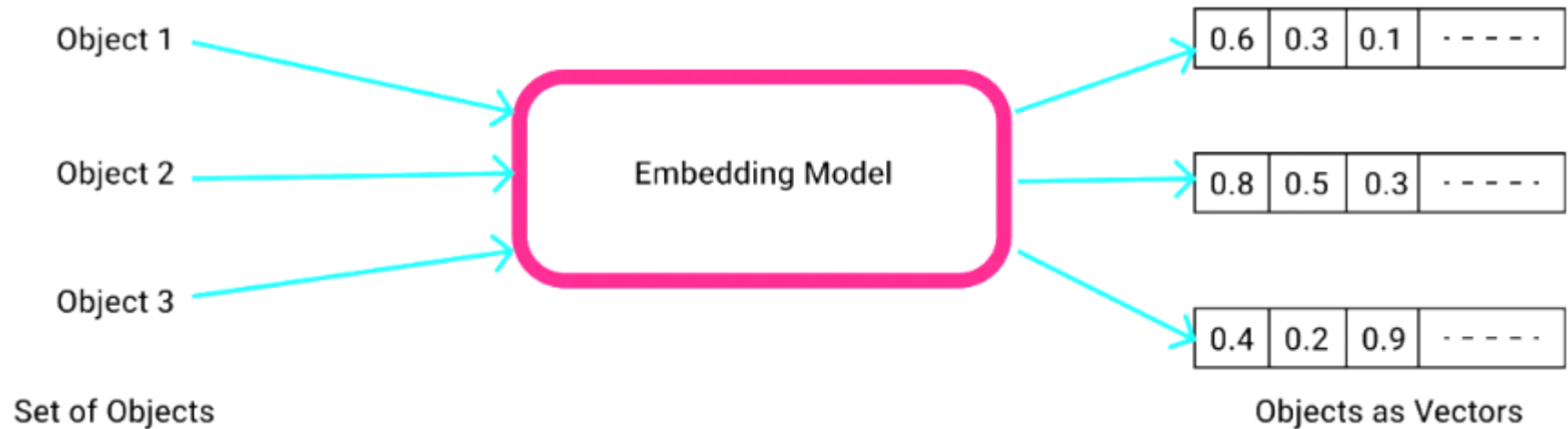
# 모델 실행
ollama run <model-name>

# 설치된 모델 목록
ollama list
```

□ Embedding

– Embedding

- 텍스트, 이미지 등의 데이터를 고차원 벡터(숫자 배열)로 변환하는 기술이다. 의미적으로 유사한 데이터는 벡터 공간에서 가까운 위치에 배치된다.



– 활용 분야

- 유사도 검색 : 의미적으로 유사한 문서 검색
- RAG : 질문과 관련된 문서 검색

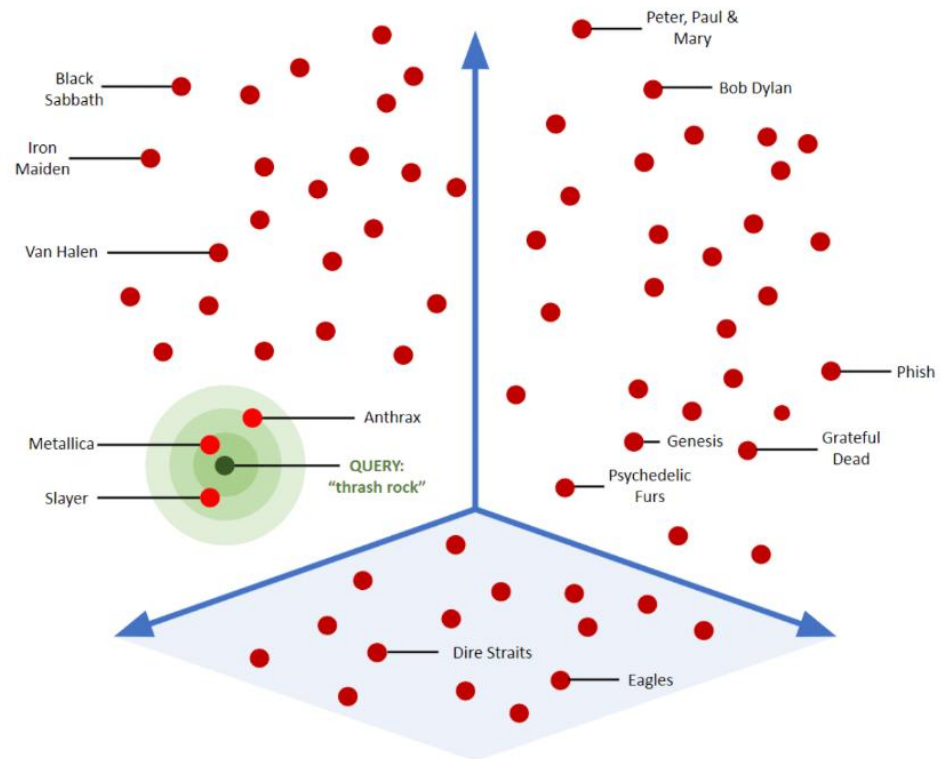
❏ Vector Store

— Vector Store

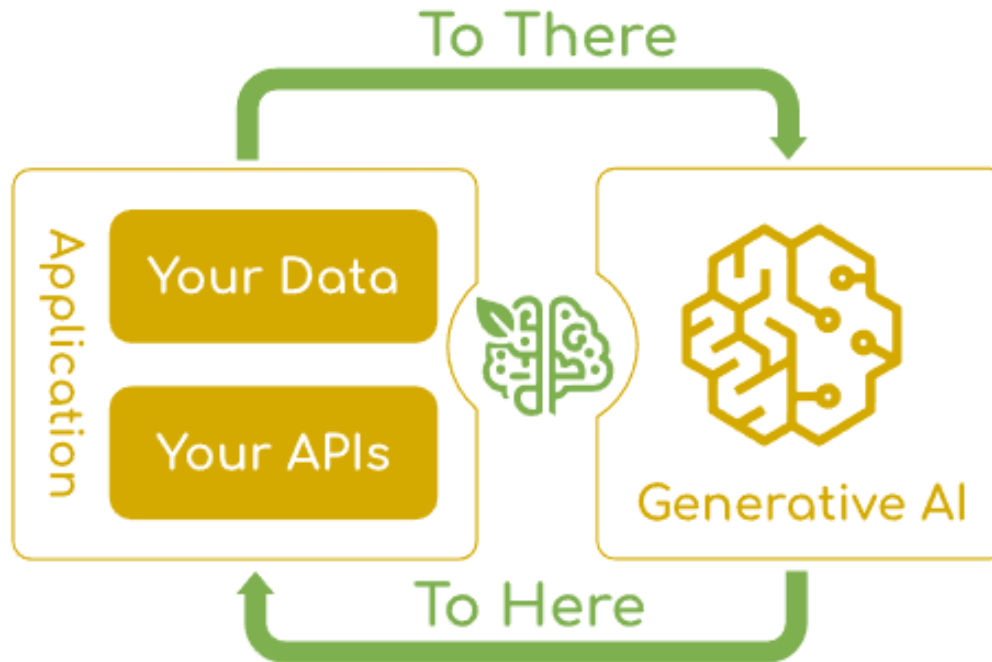
- Vector 값을 저장하고 유사도 검색을 수행할 수 있는 데이터베이스이다.
- 주요 Vector Store로는 Redis Stack, PgVector (PostgreSQL 확장), Chroma, Pinecone, Milvus 등이 있다.
- 내부적으로 유사도를 계산하여 threshold 값 이상의 문서를 반환하게 된다.

— 주요 기능

- 벡터 저장 : 벡터 데이터 저장
- 유사도 검색 : 코사인 유사도, 유클리드 거리 등이 사용
- 메타데이터 관리 : 벡터 값과 함께 출처 등을 저장



- ❑ Spring AI는 기존 Spring 생태계 내에 포함되며 AI 기능을 쉽게 통합할 수 있도록 지원하는 역할을 수행한다.
- ❑ 다양한 AI 모델 제공자에 대한 통합 인터페이스를 Spring Framework에 친화적인 방식으로 제공한다.



- ➔ Spring AI는 AI 통합의 핵심 과제인 기존 데이터와 API를 AI 모델과 연결하는 문제를 해결하는 데 중점을 둔다.
- ➔ 엔터프라이즈 환경에서 AI 기능을 자연스럽게 구현할 수 있도록 설계된 것이 Spring AI의 특징이다.

□ AI 기술의 대중화

- ChatGPT 출시 이후 AI 기술은 급속한 확산, 발전이 이루어지고 있음
- 기존 애플리케이션에 AI 기능 통합 필요성이 증가

□ Java 생태계의 필요성

- Spring AI 등장 전까지 AI 관련 프레임워크는 Python 중심으로 전개 : LangChain, LlamaIndex 등
- AI 기능의 통합을 위해서는 기술의 파편화가 불가피 : Learning Curve 및 전환비용 발생
- Java 생태계에서도 이식성과 모듈형 설계를 갖춘 유사한 프레임워크의 필요성 제기

□ 다양한 AI 모델 및 벡터 DB의 파편화

- 다양한 AI 서비스 제공 Vendor와 여러 Vector Database를 각기 다른 방식으로 연동하여야 했음
- 복잡성을 해결하기 위한 추상화 계층이 요구됨

❑ Spring AI Features

– Portable API (이식 가능한 API)

- 다양한 AI 제공자에 대한 통합 인터페이스
- 제공자 변경 시 최소한의 코드 수정
- 동일한 코드로 여러 모델 사용 가능
- OpenAI, Hugging Face, Google (Gemini), Ollama, Anthropic (Claude) 등

– Spring Native Integration

- @Configuration, @Bean을 통한 선언적 설정
- Application.yml 기반 외부화된 설정
- Auto-configuration 지원
- Spring Boot Starter 제공

– AI 개발 편의

- AI 모델 출력 결과를 POJO 매핑 (역직렬화)
- Function Calling 지원

□ 기존 방식과의 비교

- 기존 방식은 제공자 변경 시 전체적인 코드를 수정할 필요성이 존재한다

```
// OpenAI SDK 직접 사용
OpenAI client = new OpenAI(apiKey);
ChatCompletionRequest request = ChatCompletionRequest.builder()
    .model("gpt-4")
    .messages(List.of(
        new ChatMessage("user", "Hello")
    ))
    .build();
ChatCompletionResult result = client.createChatCompletion(request);
```

- Spring AI를 사용할 시 application.yml(application.properties)의 제공자 변경만으로 전환 가능
- Spring Bean으로 자동 관리가 이루어진다

```
// Spring AI 사용
@Autowired
private ChatModel chatModel; // 자동 주입

public String chat(String message) {
    return chatModel.call(message);
}
```

□ Core APIs

기능	설명	지원 제공자
Chat	텍스트 대화	OpenAI, Anthropic, Ollama 등
Embedding	텍스트 벡터화	OpenAI, Ollama, Azure 등
Image	이미지 생성	OpenAI (DALL-E), Stability AI
Audio	음성 / 텍스트	OpenAI (Whisper, TTS)
Multimodal	이미지 + 텍스트	OpenAI (GPT-4V), Anthropic (Claude)

□ Advanced Features

- Function Calling : LLM(Large Language Model)에게 Java 메서드 호출을 위임
- Structured Output : JSON 형식 응답
- Streaming : 실시간 스트리밍 응답
- RAG (Retrieval-Augmented Generation) : 검색 증강 생성

❑ Spring AI Features

– Enterprise Features

- Prompt Caching : 일부 모델 (Claude Sonnet 4.5, Opus 4.5, Haiku 4.5 등)에 적용
- Observability : Micrometer 통합 (메트릭, 트레이싱)

– Vector Store 통합

- 주요 Vector DB 지원
- Redis, Chroma, Pinecone, Weaviate, PostgreSQL (PGvector) 등
- 메타데이터 필터링 지원
- Rag (Retrieval-Augmented Generation, 검색 증강 생성) 구현을 위한 최적화

□ 활용 사례

– 대화형 AI

- 고객 지원 챗봇 : 24/7 자동 응답, FAQ 처리
- 사내 헬프데스크 : IT 문의, 인사정책 안내
- 가상 비서 : 일정 관리, 이메일 작성 보조

– 문서 기반 Q&A (RAG)

- 내부 문서 검색 및 답변 : 매뉴얼, 가이드, 정책 문서 기반 자동 응답
- 지식베이스 구축 : 노하우 축적 및 활용
- 법규 / 정책 안내 : 정부 / 공공기관 민원 자동 응답

– 콘텐츠 생성 및 자동화

- 보고서 자동 작성 : 데이터 분석 결과 요약, 주간 / 월간 보고서
- 이메일 및 공문 초안 : 상황별 템플릿 기반 자동 생성
- 코드 생성 및 리뷰 : 테스트 코드 생성, 코드 품질 검사

□ 의존성 관리 포함

- 표준프레임워크 5.0.0 부터는 `egovframe-boot-starter-parent`에 Spring AI 의존성 관리를 공식 포함하였다.

ArtifactId	관리 버전
spring-ai-advisors-vector-store	1.0.1
spring-ai-client-chat	1.0.1
spring-ai-markdown-document-reader	1.0.1
spring-ai-pdf-document-reader	1.0.1
spring-ai-rag	1.0.1
spring-ai-starter-model-chat-memory-repository-jdbc	1.0.1
spring-ai-starter-model-ollama	1.0.1
spring-ai-starter-model-transformers	1.0.1
spring-ai-starter-vector-store-pgvector	1.0.1
spring-ai-starter-vector-store-redis	1.0.1

□ 의존성 관리 포함

- parent 선언만으로 별도 BOM 없이 사용 가능하다.

```
<parent>
  <groupId>org.egovframe.boot</groupId>
  <artifactId>egovframe-boot-starter-parent</artifactId>
  <version>5.0.0</version>
  <relativePath/>
</parent>

<!-- 버전 명시 없이 바로 사용 가능 -->
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-ollama</artifactId>
</dependency>
```



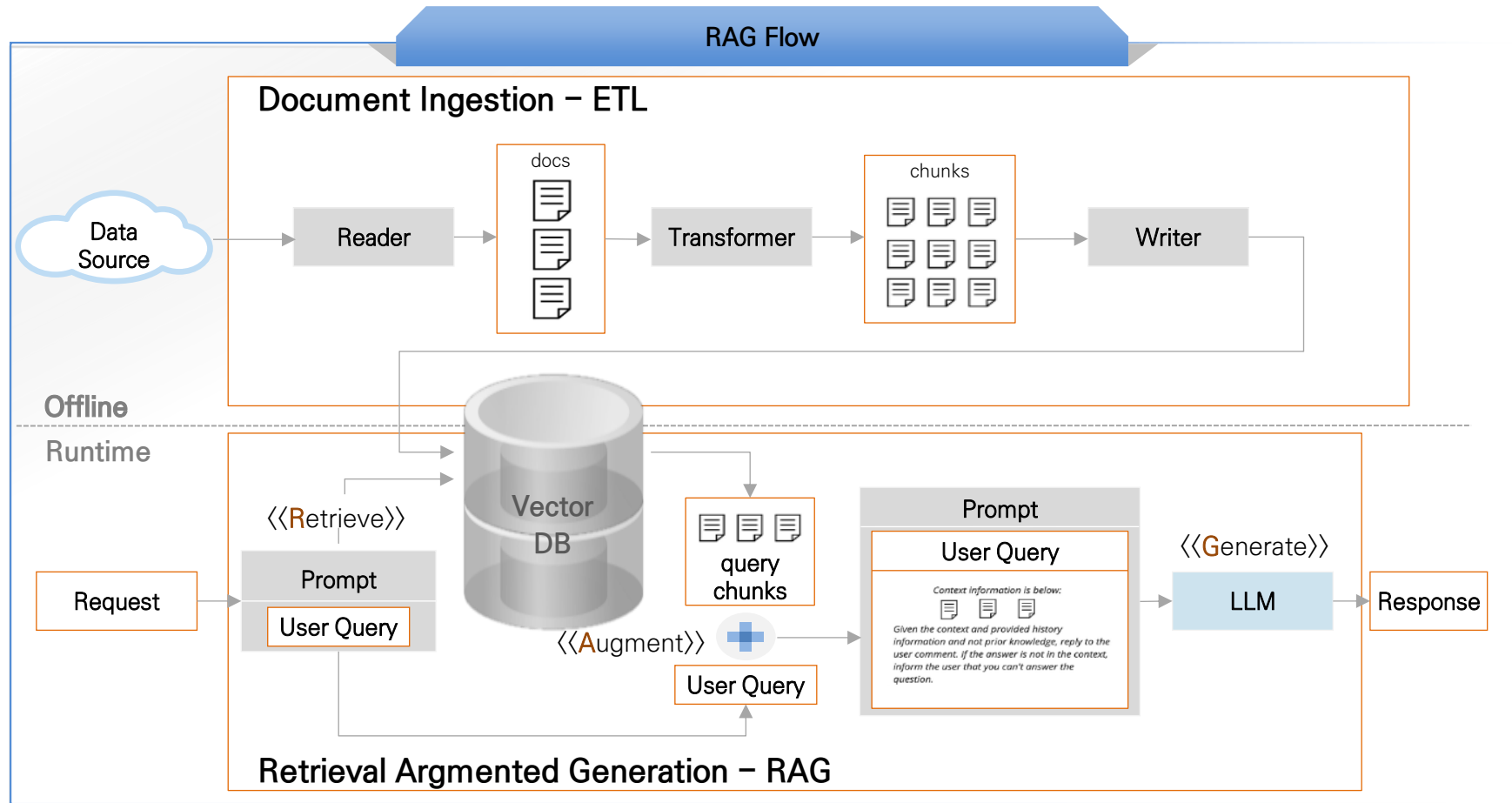

2. RAG 적용 방안 소개

1. RAG 란

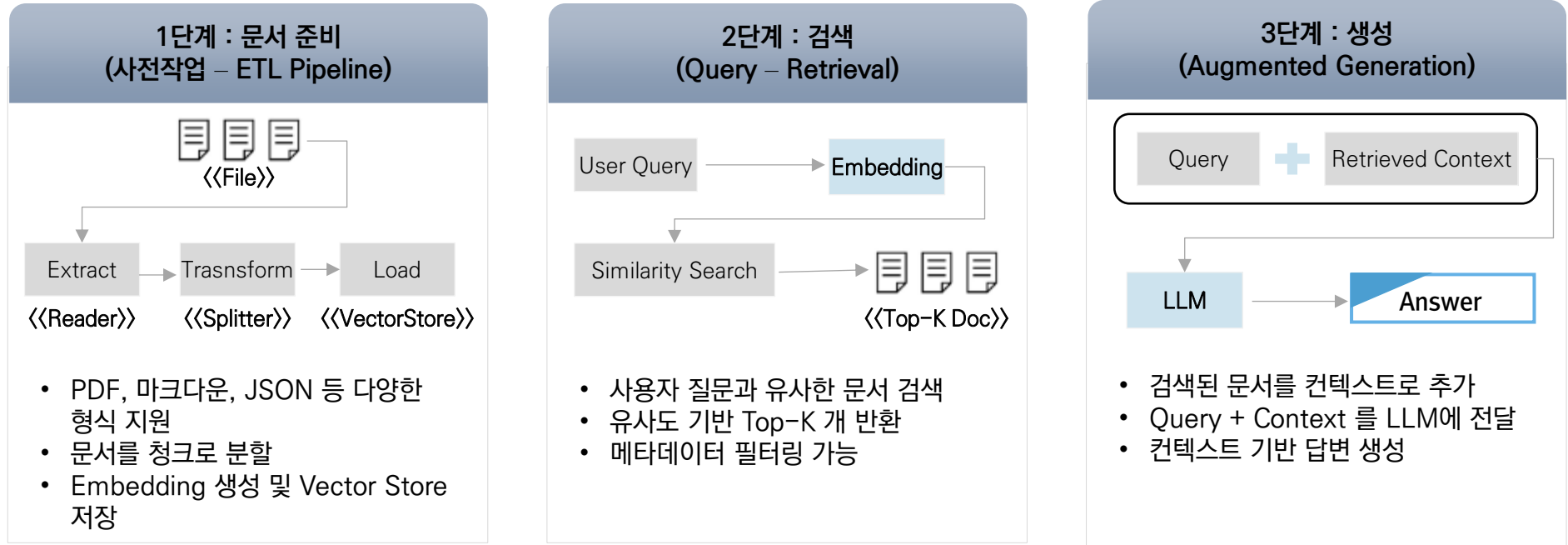
2. Advisors API

❑ RAG (Retrieval Augmented Generation)

- Retrieval (검색) / Augmented (증강) / Generation (생성)
- 외부 지식을 검색하여 AI 모델의 응답에 활용하는 기법으로, AI 모델이 가지는 환각 (Hallucination) 및 도메인 특화 지식 부족을 해결하기 위한 수단으로 사용된다.



□ RAG의 3단계 프로세스



□ RAG의 특징 및 이점

- LLM (Large Language Model) 은 기본적으로 학습 시점 이후의 데이터에 대해서는 응답이 어려움
- 특정 도메인의 심도 있는 지식을 모두 포함하기 어려움
- 학습 데이터에서 패턴을 추출하여 답변을 하지만 사실과 다를 수 있는 환각 (Hallucination) 현상이 나타날 수 있음
- Fine Tuning을 수행하려면 RAG 에 비교하여 대규모 데이터셋과 자원이 필요함

❑ Advisor

- RAG의 흐름 전체를 추상화된 API로 제공하는데 있어서 Spring AI에서 제공하는 핵심이 Advisor 이다.
- Advisor 를 사용하여 AI 기반 상호작용을 가로채서 수정 및 기능을 보완하는 동작을 수행할 수 있다.
- 반복적인 생성형 AI 패턴의 캡슐화, LLM 전송 데이터 변환, 다양한 모델 사용을 위한 이식성 제공 등에 활용 가능하다.
- 다음 코드는 ChatClient API를 사용할 경우 Advisor를 적용하는 예시이다.

```
ChatMemory chatMemory = ... // Chat Memory
VectorStore vectorStore = ... // Vector Store

var chatClient = ChatClient.builder(chatModel)
    .defaultAdvisors(
        MessageChatMemoryAdvisor.builder(chatMemory).build(), // 대화 이력을 프롬프트에 포함시키는 Advisor
        QuestionAnswerAdvisor.builder(vectorStore).build() // RAG advisor
    )
    .build();

var conversationId = "678";

String response = this.chatClient.prompt()
    // 런타임 시 Advisor의 파라미터를 설정
    .advisors(advisor -> advisor.param(ChatMemory.CONVERSATION_ID, conversationId))
    .user(userText)
    .call()
    .content();
```

❑ QuestionAnswerAdvisor

- RAG를 자동화하는 Advisor로, 문서 검색 및 컨텍스트 주입을 자동 처리할 수 있다.
- spring-ai-advisors-vector-store 모듈에 속하는 단순 RAG용 Advisor이다. 빠르게 시작할 수 있지만 커스터마이징은 제한적이다.
- ‘사용자 질문 – VectorStore 유사도 검색 – 결과를 PromptTemplate에 삽입 – LLM 호출’ 의 흐름을 갖는다.

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-advisors-vector-store</artifactId>
</dependency>
```

// 기본 사용

```
ChatClient.builder(chatModel).build().prompt()
  .advisors(QuestionAnswerAdvisor.builder(vectorStore).build())
  .user(question)
  .call().content();
```

// 커스터마이징 범위: PromptTemplate 교체 + SearchRequest 파라미터 조정

```
QuestionAnswerAdvisor.builder(vectorStore)
  .promptTemplate(customTemplate)
  .searchRequest(SearchRequest.builder().topK(5).similarityThreshold(0.7).build())
  .build();
```

// 런타임 필터 (동적 조건 검색)

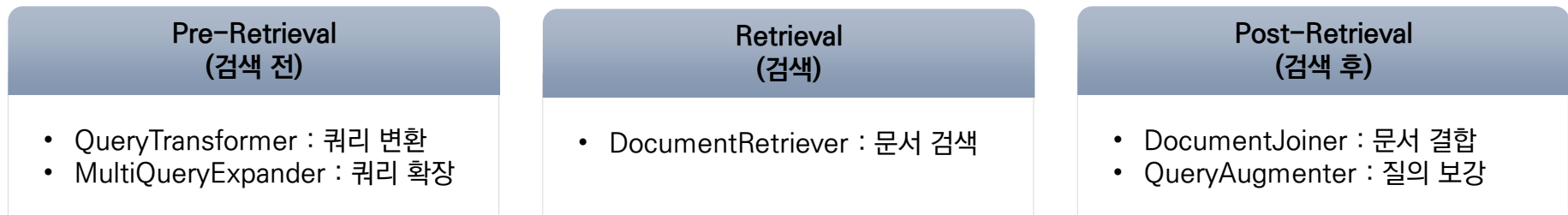
```
chatClient.prompt()
  .advisors(a -> a.param(QuestionAnswerAdvisor.FILTER_EXPRESSION, "type == 'Spring'"))
  .user(question).call().content();
```

❑ RetrievalAugmentationAdvisor

- 모듈식 아키텍처를 기반으로 하는 RAG 구현화 수행을 위한 Advisor로, 각 단계를 커스텀 가능하다는 특징이 있다.

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-rag</artifactId>
</dependency>
```

❑ Modular RAG 구조 (예시)



- 각 요소를 독립적으로 커스텀함으로써, 단순히 RAG를 수행하는 것보다 상세한 질의 요청 및 답변을 기대할 수 있다.

❑ RetrievalAugmentationAdvisor 예시

- 다음은 Chat Client API에서 RetrievalAugmentationAdvisor를 적용하여 RAG를 사용하는 기본적인 예시이다.
- 만들어진 Advisor는 기존의 코드에 삽입되어 Spring AOP(Aspect Oriented Programming)의 Advice 처럼 작동하게 된다.

```
//응답에 포함될 유사도 값을 0.5로 한 RAG Advisor 생성
Advisor ragAdvisor = RetrievalAugmentationAdvisor.builder()
    .documentRetriever(
        VectorStoreDocumentRetriever.builder()
            .similarityThreshold(0.50)
            .vectorStore(vectorStore)
            .build()
    )
    .build();

//ChatClient 호출 코드 내에서 삽입되어 동작
String answer = chatClient.prompt()
    .advisors(ragAdvisor)
    .user("Spring AI란?")
    .call()
    .content();
```

❑ Modular RAG Custom (1/4)

- RAG 수행 전에 RewriteQueryTransformer를 사용하는 예시이다.
- RewriteQueryTransformer는 LLM을 활용하여 사용자 쿼리를 재작성해서 사용자 쿼리가 모호하거나 장황한 경우 유용하다.

```
// LLM을 사용하여 쿼리를 자동으로 재작성
Advisor ragAdvisor = RetrievalAugmentationAdvisor.builder()
    .queryTransformers(
        RewriteQueryTransformer.builder()
            .chatClientBuilder(chatClientBuilder)
            .build()
    )
    .documentRetriever(
        VectorStoreDocumentRetriever.builder()
            .similarityThreshold(0.50)
            .vectorStore(vectorStore)
            .build()
    )
    .build();
```


❑ Modular RAG Custom (2/4)

- CompressionQueryTransformer는 LLM을 활용하여 대화 기록과 후속 쿼리를 독립형 쿼리로 압축한다.
- 대화 기록이 길고 후속 쿼리가 대화 맥락과 관련될 경우 유용하게 사용될 수 있다.

```
// 사용자 쿼리를 대화 기록을 참조하여 독립 쿼리로 변환
Query query = Query.builder()
    .text("And what is its second largest city?")
    .history(new UserMessage("What is the capital of Denmark?"),
        new AssistantMessage("Copenhagen is the capital of Denmark."))
    .build();

QueryTransformer queryTransformer = CompressionQueryTransformer.builder()
    .chatClientBuilder(chatClientBuilder)
    .build();

Query transformedQuery = queryTransformer.transform(query);
```

❑ Modular RAG Custom (3/4)

- TranslationQueryTransformer는 LLM을 활용하여 쿼리를 지정 언어로 변환한다
- LLM이 특정 언어로 훈련되었고, 사용자 쿼리가 다른 언어인 경우 유용하게 사용된다.

```
// 덴마크어 질의를 영어로 번역해서 LLM에 전달
Query query = new Query("Hvad er Danmarks hovedstad?");

QueryTransformer queryTransformer = TranslationQueryTransformer.builder()
    .chatClientBuilder(chatClientBuilder)
    .targetLanguage("english")
    .build();

Query transformedQuery = queryTransformer.transform(query);
```

❑ Modular RAG Custom (4/4)

- 쿼리 변환 외에도 여러 커스텀 설정이 가능하다.

```
// Empty Context 허용 설정
.queryAugmenter(
    ContextualQueryAugmenter.builder()
        .allowEmptyContext(true) // 검색 결과 없어도 진행
        .build()
)

// 메타데이터 필터링
VectorStoreDocumentRetriever retriever =
    VectorStoreDocumentRetriever.builder()
        .vectorStore(vectorStore)
        .filterExpression("category == 'framework'")
        .build();
```



3. Langchain4j와 SpringAI 비교

1. Langchain4j 란
2. Spring AI와의 비교
3. 표준프레임워크의 Langchain4j 공식 지원

❑ Langchain4j

- Java 용 LLM 통합 프레임워크
- Python의 Langchain을 Java 생태계에 맞게 재설계하는 것을 목적으로 등장하였다.
- 2023년 출시, 버전 1.0.0 정식 출시 (2025년 기준)

❑ 핵심 특징

- 널리 사용되는 LLM과 벡터 데이터베이스에 대한 접근을 제공한다.
- 다양한 엔터프라이즈 자바 프레임워크와의 통합을 제공한다.
- AIServices 기반 프록시 패턴을 통해 선언적 인터페이스 기반으로 AI 서비스를 구현한다.

```
// 인터페이스 정의
public interface RagChatbot {
    @SystemMessage("당신은 지식 기반 질의응답 시스템입니다...")
    Flux<String> streamChat(@UserMessage String query);
}
```

```
// LangChain4j가 런타임에 구현체 자동 생성
RagChatbot bot = AIServices.builder(RagChatbot.class)
    .streamingChatModel(model)
    .contentRetriever(retriever) // RAG 자동 적용
    .chatMemory(memory) // 메모리 자동 관리
    .build();
```

❑ **AI Services Reflection 기반 프록시**

- Langchain4j의 가장 큰 특징
- Spring Data JPA 에서 Repository 인터페이스만 정의하면 구현체가 자동 생성되는 것과 동일한 패턴이다.

```
// Spring Data JPA
interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByName(String name); // 메서드 시그니처만 정의
}
// Spring이 프록시 객체를 자동 생성하여 SQL 실행
```

```
// Langchain4j AI Services
interface RagChatbot {
    @SystemMessage("당신은 도움이 되는 AI입니다")
    String chat(@UserMessage String query); // 메서드 시그니처만 정의
}
// LangChain4j가 프록시 객체를 자동 생성하여 LLM 호출
```

- 인터페이스만 정의하면 Langchain4j가 런타임에 구현체를 생성한다.
- ‘@SystemMessage’, ‘@UserMessage’등의 어노테이션으로 프롬프트를 자동 구성하게 된다.
- 설정 코드가 간결해지게 된다는 장점을 가진다.

□ 설정 방식 비교

구 분	Spring AI	Langchain4j
핵심 패턴	Advisor Chain	AIService Reflection 기반 프록시
구현 방식	ChatClient + Advisor Chain	인터페이스 + 어노테이션
사용자 메시지	.user("...") 메서드	@UserMessage 어노테이션
RAG 적용	RetrievalAugmentationAdvisor	contentRetriever() 빌더
Chat Memory	MessageChatMemoryAdvisor	chatMemory() 빌더

❑ Spring AI – Advisor Chain 패턴

```
// Advisor들을 명시적으로 체인으로 연결
ChatClient chatClient = ChatClient.builder(chatModel)
    .advisors(
        new MessageChatMemoryAdvisor(chatMemory),
        new RetrievalAugmentationAdvisor(documentRetriever)
    )
    .build();

// 사용
String response = chatClient
    .prompt()
    .system("당신은 도움이 되는 AI입니다")
    .user(userQuery)
    .advisors(a -> a.param(CONVERSATION_ID, sessionId))
    .call()
    .content();
```

➔ 명시적으로 Advisor들을 연결하고, 각 Advisor가 순차적으로 처리한다.

❑ Langchain4j – AIServices 프록시 패턴

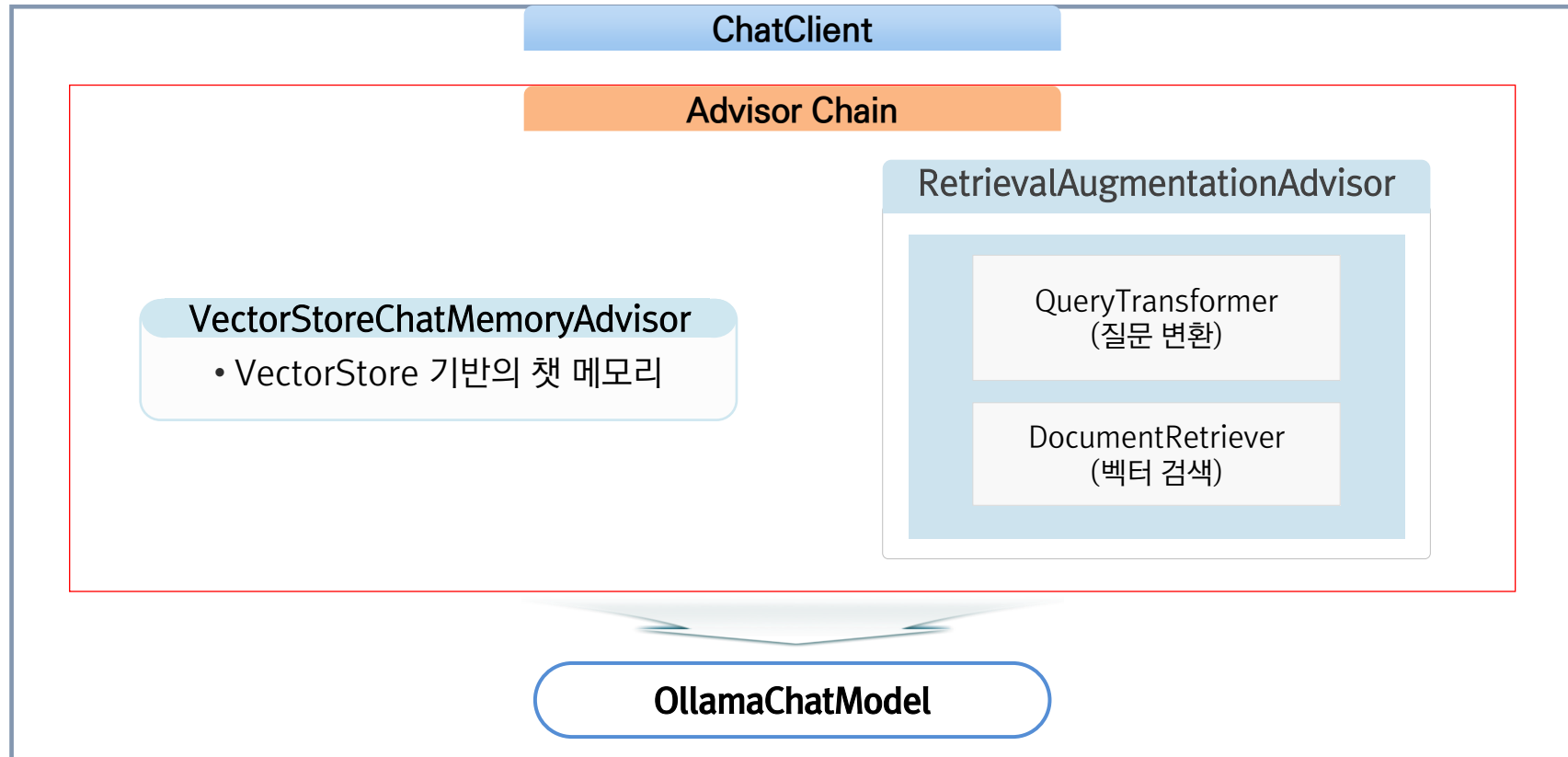
```
// 1단계: 인터페이스 정의
interface RagChatbot {
    @SystemMessage("당신은 도움이 되는 AI입니다")
    String chat(@UserMessage String query);
}

// 2단계: 프록시 생성 - RAG와 메모리 자동 적용
RagChatbot bot = AIServices.builder(RagChatbot.class)
    .chatModel(model)
    .contentRetriever(retriever) // RAG 자동 적용
    .chatMemory(memory) // Chat Memory 자동 적용
    .build();

// 3단계: 사용 - 인터페이스 메서드 호출만
String response = bot.chat(userQuery);
```

➔ 빌더에 retriever 와 memory를 전달하면 프록시가 자동으로 RAG와 Chat Memory 관리를 처리하게 된다.

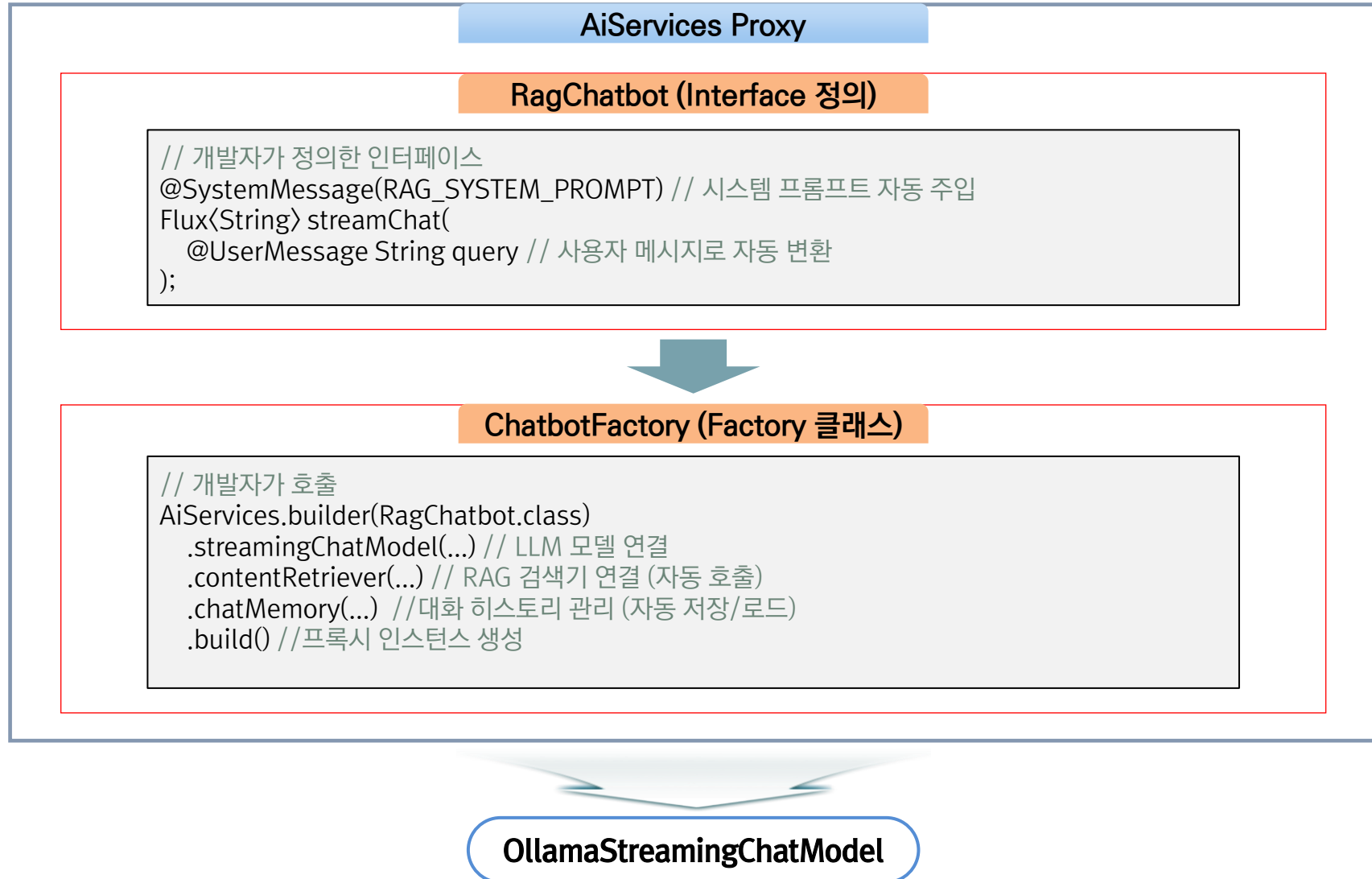
❑ Spring AI 아키텍처 (RAG)



→ Spring Ai 에서는 RAG 구현에 있어서 Advisor 체인 패턴이 사용된다.

→ 'advisors()' 메서드로 동적 조합을 수행한다.

❑ Langchain4j 아키텍처 (RAG)



□ 의존성 관리 포함

- 표준프레임워크 5.0.0 부터는 `egovframe-boot-starter-parent`에 Langchain4j 의존성 관리를 공식 포함하였다.

구분	ArtifactId	관리 버전
정식 릴리즈	langchain4j	1.8.0
정식 릴리즈	langchain4j-ollama	1.8.0
정식 릴리즈	langchain4j-redis	1.8.0
정식 릴리즈	langchain4j-spring-boot-starter	1.8.0
Beta	langchain4j-pgvector	1.8.0-beta15
Beta	langchain4j-embeddings	1.8.0-beta15
Beta	langchain4j-reactor	1.8.0-beta15
Beta	langchain4j-easy-rag	1.8.0-beta15
Beta	langchain4j-document-parser-apache-pdfbox	1.8.0-beta15

□ 의존성 관리 포함

- parent 선언만으로 별도 BOM 없이 사용 가능하다.

```
<parent>
  <groupId>org.egovframe.boot</groupId>
  <artifactId>egovframe-boot-starter-parent</artifactId>
  <version>5.0.0</version>
  <relativePath/>
</parent>

<!-- 버전 명시 없이 바로 사용 가능 -->
<dependency>
  <groupId>dev.langchain4j</groupId>
  <artifactId>langchain4j</artifactId>
</dependency>
```

➔ 표준프레임워크 5.0.0부터는 Spring AI와 LangChain4j를 모두 공식 지원하여, 프로젝트 요구사항에 따라 선택할 수 있다.



4. SpringAI 기반 RAG Chatbot 구현 사례

1. RAG Chatbot 소개
2. 구성 및 특징
3. 동작 시연

□ 특징 및 주의사항

- RAG(Retrieval-Augmented Generation) 기반의 AI 질의응답 시스템 샘플 프로젝트이다
- Spring AI의 **ChatClient + Advisor 패턴**을 사용하여 RAG 시스템을 구현하였다. Advisor를 조합하여 채팅 메모리, RAG 검색, 질의 압축 등의 기능을 선언적으로 구성한다.
- 로컬의 LLM은 Ollama를 사용하여 설치 또는 Huggingface에서 사전에 현재 하드웨어 사양에 맞추어 준비하는 방법을 사용 가능하다.
- Embedding 작업은 Huggingface에 배포되는 여러 모델 중 적합한 모델을 취사 선택하여 Onnx로 변환 후, 로컬에서 사용한다.
- 인덱싱 가능한 문서의 종류는 마크다운 파일과 PDF 파일로 구성되어 있다.
- Vector Database는 Redis Stack을 사용하며, Redis Stack은 Apache 2.0 라이선스가 **아닌** RSALv2 / SSPLv1 의 듀얼 라이선스이므로 상용 서비스 제공 시에는 라이선스 조항을 확인하여야 한다.

❑ src/main/java

- 클래스, 인터페이스 등 자바 파일이 위치
- config 패키지에서 ETL / Chat Memory / RAG / 쿼리 압축 관련 설정 사항을 확인 가능

❑ src/main/resources

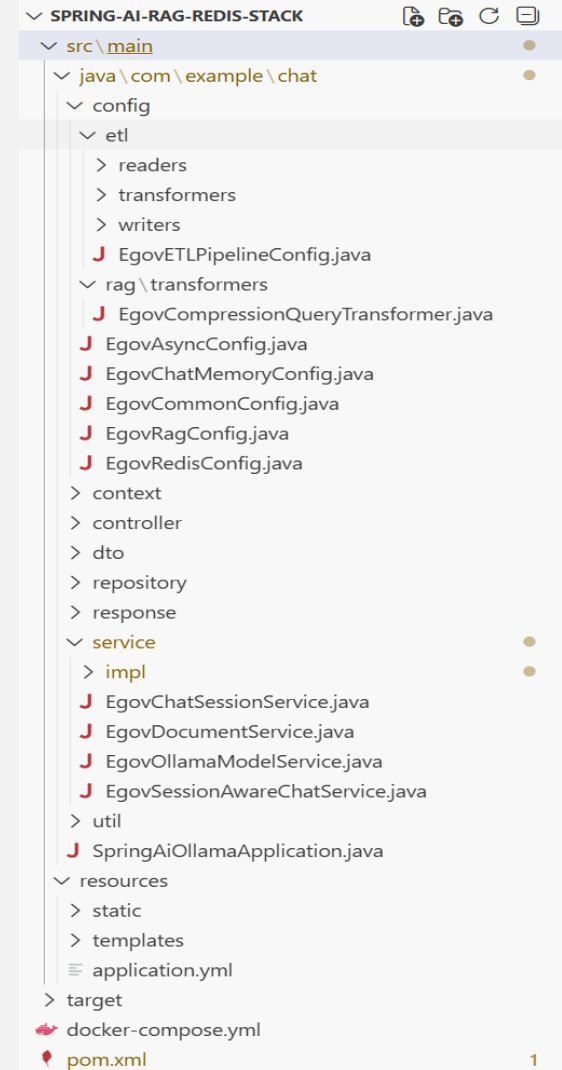
- static : js, css, image 등의 정적 파일들이 위치
- templates : Thymeleaf 가 사용된 View Template 파일들이 위치
- application.yml : 애플리케이션 전반의 설정이 명시되어 있는 파일이다.
- Embedding 용 Model은 resources/model 내에 위치하여야 한다 (초기 설정 기준)

❑ Pom.xml

- Maven 빌드 정보를 기재

❑ Application.yml

- Ollama / Redis Stack 기본 사항 외에 추가 기능 설정들을 이 파일 내에서 지정 가능하다.
- RAG 임계값, TopK 값, Chunk 분할, 문서 경로, 키워드 추출, 요약 정리 설정 등이 있다.

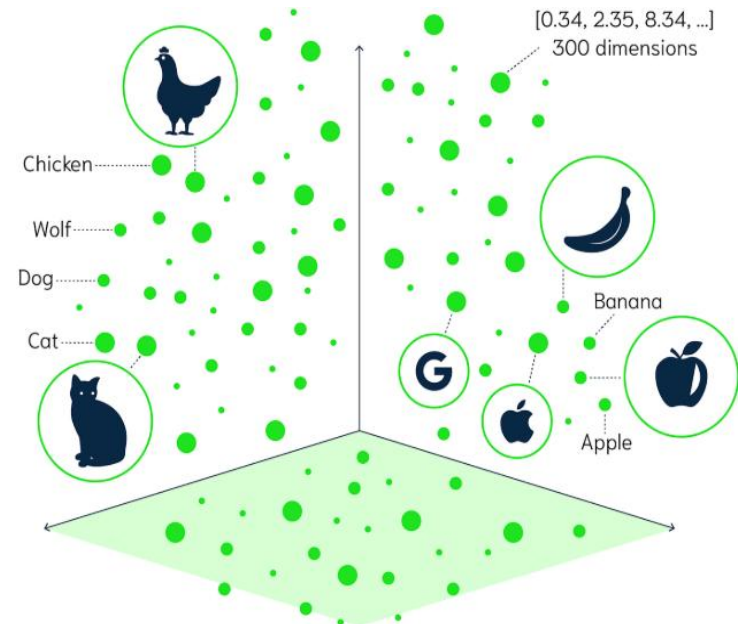


❑ Onnx (Open Neural Network Exchange)

- Onnx (Open Neural Network Exchange)는 기계학습 모델을 다른 딥러닝 프레임워크 환경 (ex. Tensorflow, PyTorch, etc..)에서 서로 호환되게 사용할 수 있도록 만들어진 공유 플랫폼이다
- 다양한 프레임워크 간의 호환성 문제를 해결하고, 모델 배포 및 활용에 유연성을 제공할 수 있다.

❑ Embedding

- 자연어를 기계가 이해할 수 있는 숫자의 나열인 Vector로 바꾸는 과정을 의미한다.
- Vector로 간 거리를 계산하여 제일 가까운 값을 가진 문서를 제일 유사한 문서라고 판단한다.
- Embedding에 사용되는 모델은 Huggingface에서 배포되는 model을 이용 가능.



❑ Onnx 형식의 모델 익스포트 (1/3)

- Embedding 수행은 Huggingface에서 공개되는 Embedding 모델을 직접적으로 호출하는 것도 가능
- 단, 호출 회수의 한계가 존재하므로 많은 양의 데이터를 처리하는 것은 불가능하다.
- Huggingface 에 배포되는 여러 모델 중 적합한 모델을 취사 선택하여 Onnx (Open Neural Network Exchange)로 변환 후, 로컬에서 사용하는 것이 가능하다.
- Python에서 가상 환경을 설정하고 필요한 패키지를 설치하며, 모델을 Onnx 포맷으로 익스포트한다.

❑ Onnx 형식의 모델 익스포트 (2/3)

- 윈도우 환경에서 Embedding 모델을 Onnx로 익스포트하는 예시이다.

1. 가상 환경 생성

```
python -m venv venv
```

2. 가상 환경 활성화

```
.\venv\Scripts\activate
```

3. pip 업그레이드

```
pip install --upgrade pip
```

4. 필요한 패키지를 묶어서 설치 (중요: 순서 및 묶음 설치)

```
pip install -U huggingface-hub transformers optimum[onnx] onnxruntime sentence-transformers
```

5. (선택사항) 설치된 패키지 버전 확인

```
pip show huggingface-hub transformers optimum onnxruntime sentence-transformers
```

6. (선택사항) HuggingFace 로그인 (일부 모델에서 필요할 수 있음)

```
hf auth login
```

7. 모델을 ONNX 포맷으로 익스포트

```
optimum-cli export onnx --model [모델명] [export 경로]
```

```
# 예: optimum-cli export onnx --model google/embeddinggemma-300m .
```

❑ Onnx 형식의 모델 익스포트 (3/3)

- macOS 환경에서 Embedding 모델을 Onnx로 익스포트하는 예시이다.

1. 가상 환경 생성

```
python3 -m venv venv
```

2. 가상 환경 활성화

```
source ./venv/bin/activate
```

3. pip 업그레이드

```
pip install --upgrade pip
```

4. 필요한 패키지를 묶어서 설치 (중요: 순서 및 묶음 설치)

```
pip install -U huggingface-hub transformers optimum[onnx] onnxruntime sentence-transformers
```

5. (선택사항) 설치된 패키지 버전 확인

```
pip show huggingface-hub transformers optimum onnxruntime sentence-transformers
```

6. (선택사항) HuggingFace 로그인 (일부 모델에서 필요할 수 있음)

```
hf auth login
```

7. 모델을 ONNX 포맷으로 익스포트

```
optimum-cli export onnx --model [모델명] [export 경로]
```

```
# 예: optimum-cli export onnx --model google/embeddinggemma-300m .
```

□ 일반 질의 (1/2)

- RAG를 사용하지 않고 LLM에만 의존하여 일반적인 질의응답을 수행하는 화면이다.
- 질의에 사용할 모델은 상단 셀렉트 박스에서 변경이 가능하다.

RAG 채팅

일반 채팅

사용할 모델:

qwen3-8b:Q4_K_M

선택된 모델: qwen3-8b:Q4_K_M

2026년은 한국의 12간지 중 어떤 동물의 해인지 알려 줘

Okay, the user is asking which animal's year 2026 is in the Chinese zodiac. Let me recall the cycle. The Chinese zodiac has 12 animals, each representing a year in a 12-year cycle. The order is Rat, Ox, Tiger, Rabbit, Dragon, Snake, Horse, Goat, Monkey, Rooster, Dog, Pig, and then it repeats.

First, I need to figure out what year 2026 corresponds to. I know that 2020 was the Year of the Rat. Then 2021 was the Ox, 2022 Tiger, 2023 Rabbit, 2024 Dragon, 2025 Snake, 2026 Horse. Wait, that seems right. Let me check again. 2020: Rat, 2021: Ox, 2022: Tiger, 2023: Rabbit, 2024: Dragon, 2025: Snake, 2026: Horse. Yes, that's correct. So 2026 is the Year of the Horse. But wait, I should make sure. Maybe I should cross-verify with another method. For example, if 2020 is Rat, then adding 12 years would bring us back to Rat again. So $2020 + 12 = 2032$. So 2026 is 6 years after 2020. 2020 (Rat), 2021 (Ox), 2022 (Tiger), 2023 (Rabbit), 2024 (Dragon), 2025 (Snake), 2026 (Horse). Yep, that's correct. So the answer should be the Horse. But wait, I think I might have confused the order. Let me list them again in order: 1. Rat, 2. Ox, 3. Tiger, 4. Rabbit, 5. Dragon, 6. Snake, 7. Horse, 8. Goat, 9. Monkey, 10. Rooster, 11. Dog, 12. Pig. So 2020 is Rat (1), 2021 Ox (2), 2022 Tiger (3), 2023 Rabbit (4), 2024 Dragon (5), 2025 Snake (6), 2026 Horse (7). So yes, 2026 is the Horse. I think that's right. But maybe I should check an online source or a calendar to confirm. However, since I can't access external resources, I'll rely on my knowledge. The Chinese zodiac cycles every 12 years, so 2026 is indeed the Year of the Horse. Therefore, the answer is the Horse.

2026년은 한국의 12간지(조선시대의 12년간지) 중 말의 해입니다.

□ 일반 질의 (2/2)

- RAG를 사용하지 않고 LLM에만 의존하여 일반적인 질의응답을 수행하는 화면이다.
- 질의에 사용할 모델은 상단 셀렉트 박스에서 변경이 가능하다.

12간지는 다음과 같은 순서로 순환합니다:

1. 쥐 (2020년, 2012년, 2000년, ...)
2. 소 (2021년, 2013년, 2001년, ...)
3. 호랑이 (2022년, 2014년, 2002년, ...)
4. 토끼 (2023년, 2015년, 2003년, ...)
5. 용 (2024년, 2016년, 2004년, ...)
6. 사자 (2025년, 2017년, 2005년, ...)
7. 말 (2026년, 2018년, 2006년, ...)

따라서 2026년은 말의 해입니다.

메시지를 입력하세요...

전송

❑ RAG 질의 (1/6)

- RAG를 사용하여 문서의 내용을 LLM에 컨텍스트로 제공하여 질의응답을 수행하는 화면이다.
- Qwen과 같은 추론형 모델을 사용 시, 추론 과정에서 컨텍스트를 참고하였다는 것을 직접적으로 확인 가능하다.

RAG 채팅

일반 채팅

사용할 모델:

qwen3-8b:Q4_K_M

선택된 모델: qwen3-8b:Q4_K_M

표준프레임워크에서 aop 구현은 어떻게 가이드 되는지 알려 줘

Okay, let's see. The user is asking about how AOP is guided in a standard framework, specifically referring to the context provided.

First, I need to parse the context information. The context mentions several points about AOP in the context of a common base layer. It talks about applying AOP for exception handling and transaction management in the execution environment. There's a mention of XML schema support for AOP, defining aspects, advice, and pointcuts. The example given includes a test class with an annotation aspect and the order of advice execution.

The user wants to know how AOP is guided in a standard framework. From the context, it seems that the framework uses XML configuration (like context-aspect.xml) to define aspects, advice, and pointcuts. They also mention the order of advice execution, such as before, around, after, and afterThrowing. Additionally, there's a note about Spring version differences affecting the order.

The answer should focus on the guidelines provided in the context. Since the context doesn't mention other frameworks like AspectJ or others, it's safe to stick to the information given. The key points are the use of XML for configuration, defining aspects with advice and pointcuts, handling exceptions and transactions, and the execution order of advice. Also, the note about Spring versions affecting the order is important.

메시지를 입력하세요...

전송

❑ RAG 질의 (2/6)

- RAG 수행 중의 터미널 콘솔 화면이다.
- 사전에 application.yml에서는 임계값을 0.70, topK 값을 3으로 설정하여 둔 상태이다.
- 로그를 확인해 보면 socre 0.8 수준의 문서가 3건 검색된 것으로 확인된다.

```
2026-03-27T10:28:01.720+09:00 INFO 30748 --- [spring-ai-rag-redis] [oundedElastic-2] .c.c.r.t.EgovCompressionQueryTransformer : 최종 압축된 질문: '표준프레임워크에서 aop 구현은 어떻게 가이드 되는지 알려 줘'
2026-03-27T10:28:01.720+09:00 INFO 30748 --- [spring-ai-rag-redis] [oundedElastic-2] com.example.chat.config.EgovRagConfig : SessionAwareQueryTransformer 완료 - 압축된 질문: '표준프레임워크에서 aop 구현은 어떻게 가이드 되는지 알려 줘'
2026-03-27T10:28:01.720+09:00 INFO 30748 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : RAG 문서 검색 시작 - 질문: '표준프레임워크에서 aop 구현은 어떻게 가이드 되는지 알려 줘'
2026-03-27T10:28:05.194+09:00 INFO 30748 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : RAG 문서 검색 완료 - 검색된 문서 수: 3 (모두 LLM 컨텍스트에 포함)
2026-03-27T10:28:05.194+09:00 INFO 30748 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : 문서 #1: 유사도=0.8885 (distance=0.1115), 길이=228자, 내용=3. AOP - 설명 (15/17) 2. 공통기반 레이어 실행환경 AOP 가이드라인 실행환경은 예외 처리와 트랜잭션 처리에 AOP를 적용함 실행환경 AOP 가이드라인 예외 처리...
2026-03-27T10:28:05.194+09:00 INFO 30748 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : 문서 #2: 유사도=0.8804 (distance=0.1196), 길이=136자, 내용=3. AOP - 설명 (17/17) 2. 공통기반 레이어 실행환경 AOP 가이드라인 트랜잭션 처리 관점(Aspect) 정의: resources/egovframework.sprin...
2026-03-27T10:28:05.194+09:00 INFO 30748 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : 문서 #3: 유사도=0.8778 (distance=0.1222), 길이=352자, 내용=3. AOP - 설명 (9/17) 2. 공통기반 레이어 XML 스키마를 이용한 AOP 지원 (8/11) Aspect 실행하기 정상 실행의 경우 public class Annota...
2026-03-27T10:28:05.194+09:00 INFO 30748 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : 총 716자의 컨텍스트가 LLM에 전달됩니다
```


❑ RAG 질의 (3/6)

- 제공된 컨텍스트를 기반으로 LLM이 답변을 작성하여 반환한 화면이다.
- 사전에 제공한 문서의 내용을 정확히 반영한 것을 확인할 수 있다.

I need to make sure not to include any external knowledge. The answer should be based solely on the provided context. So, the guidelines include XML-based configuration, specific advice types, and the order of execution. Also, the example given shows how aspects are applied in a test case, which might be part of the framework's guidance.

표준프레임워크에서 AOP 구현 가이드라인은 XML 스키마를 이용한 구성과 관점(Aspect) 정의를 통해 제공됩니다. 예외 처리와 트랜잭션 처리에 AOP를 적용하며, Advice(예: before, around, after, afterThrowing)의 실행 순서는 대상 메소드 수행 전 후에 따라 달라집니다. 구체적으로, before → around → 대상 메소드 → after(finally) → afterThrowing 순으로 실행되며, Spring 버전에 따라 순서가 다를 수 있습니다. 또한, 공통기반 레이어에서 Aspect 정의 시 Pointcut과 JoinPoint를 명시적으로 설정해야 하며, XML 파일(예: context-aspect.xml)을 통해 Advice를 적용합니다.

메시지를 입력하세요...

전송

❑ RAG 질의 (4/6)

- RAG를 사용한 추가 질의 화면이다.
- 첫 질문과 마찬가지로 LLM이 문서 내용을 컨텍스트로 참조하고 있음을 확인 가능하다.

사용할 모델: qwen3-8b:Q4_K_M ▼ 선택된 모델: qwen3-8b:Q4_K_M

각 advise의 예시를 간단하게 제시해 줘

Okay, let's see. The user is asking for examples of each advice type in Spring AOP based on the provided context. Let me go through the context information again to make sure I understand what's needed.

First, the context mentions different types of advice: Before, After returning, After throwing, After (finally), and Around. Each has a brief description. The user wants a simple example for each.

Looking at the context, there's a mention of After (finally) advice being used for resource cleanup, which is a common use case. The example given is a class with a method that prints a message when the target method is executed. But that's just one example. The user probably wants a quick example for each advice type.

For Before advice, the example would be something like logging before a method is called. The context mentions that before advice runs before the join point. So maybe a method that logs "Method is about to execute" before the target method.

After returning advice would be used after the method successfully returns. An example could be logging the result or handling the return value. The context says it's used when the join point returns successfully.

메시지를 입력하세요

전송

□ RAG 질의 (5/6)

- 추가 질의에 대한 RAG 수행 중의 터미널 콘솔 화면이다.
- 이전과 마찬가지로 문서를 검색하여 결과를 컨텍스트로 LLM에 전달하게 된다.

```

2026-03-27T10:35:06.468+09:00 INFO 24152 --- [spring-ai-rag-redis] [oundedElastic-2] com.example.chat.config.EgovRagConfig : SessionAwareQueryTransformer 완료 - 압축
된 질문: '표준프레임워크에서 AOP의 예외 처리 및 트랜잭션 처리 어드바이스 예시를 간단히 설명해 줘'
2026-03-27T10:35:06.470+09:00 INFO 24152 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : RAG 문서 검색 시작 - 질문: '표준프레임워
크에서 AOP의 예외 처리 및 트랜잭션 처리 어드바이스 예시를 간단히 설명해 줘'
2026-03-27T10:35:06.508+09:00 INFO 24152 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : RAG 문서 검색 완료 - 검색된 문서 수: 3 (
모두 LLM 컨텍스트에 포함)
2026-03-27T10:35:06.509+09:00 INFO 24152 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : 문서 #1: 유사도=0.9088 (distance=0.0912
), 길이=228자, 내용=3. AOP - 설명 (15/17) 2. 공통기반 레이어 실행환경 AOP 가이드라인 실행환경은 예외 처리와 트랜잭션 처리에 AOP를 적용함 실행환경 AOP 가
이드라인 예외 처
리...
2026-03-27T10:35:06.509+09:00 INFO 24152 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : 문서 #2: 유사도=0.8982 (distance=0.1018
), 길이=3737자, 내용=--- linkTitle: "AOP 가이드라인" title: 실행환경 AOP 가이드라인 url: "/egovframe-runtime/foundation-layer-core/aop/...
2026-03-27T10:35:06.509+09:00 INFO 24152 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : 문서 #3: 유사도=0.8975 (distance=0.1025
), 길이=136자, 내용=3. AOP - 설명 (17/17) 2. 공통기반 레이어 실행환경 AOP 가이드라인 트랜잭션 처리 관점(Aspect) 정의: resources/egovframework.sprin...
2026-03-27T10:35:06.510+09:00 INFO 24152 --- [spring-ai-rag-redis] [ ai-advisor-1] com.example.chat.config.EgovRagConfig : → 총 4101자의 컨텍스트가 LLM에 전달됩니다
  
```

□ RAG 질의 (6/6)

- 추가 질의에 대해 제공된 컨텍스트를 기반으로 LLM이 답변을 작성하여 반환한 화면이다.
- 문서 내용을 기반으로 답변이 작성된 것을 확인 가능하다.

Alright, compiling all that into a list of examples for each advice type.

- **Before Advice:** 메소드 실행 전 로그 기록.
예: `public void beforeMethod(JoinPoint joinPoint) { System.out.println("메소드 시작 전"); }`
- **After Returning Advice:** 메소드 정상 종료 후 처리.
예: `public void afterReturn(JoinPoint joinPoint, Object result) { System.out.println("메소드 결과: " + result); }`
- **After Throwing Advice:** 예외 발생 시 처리.
예: `public void afterThrowing(JoinPoint joinPoint, Throwable ex) { System.out.println("예외 발생: " + ex.getMessage()); }`
- **After (Finally) Advice:** 메소드 종료 후 무조건 실행 (리소스 해제 등).
예: `public void afterFinally(JoinPoint joinPoint) { System.out.println("메소드 종료 후 처리"); }`
- **Around Advice:** 메소드 전후로 컨트롤.
예: `public Object aroundMethod(ProceedingJoinPoint pjp) throws Throwable { System.out.println("메소드 시작"); Object result = pjp.proceed(); System.out.println("메소드 종료"); return result; }`

메시지를 입력하세요...

전송

감사합니다.